

1. Introduction

This assessment includes analyzing conventional problem called N-body problem in computational science via distance calculation simulation with $O(N^2)$. The objective is to identify the nearest neighbor and the furthest neighbor in terms of sets of 100,000 points distributed in two-dimensional space. Specifically, it compares that the topological difference between standard geometry and wraparound geometry affects the distance distribution. Furthermore, the performance of the optimized algorithm utilizing naïve algorithm and symmetry is evaluated by applying parallelization method using OpenMP and demonstrate the impact of the various scheduling strategies such as Static, Dynamic and Guided on execution time and load balancing.

2. Preliminary

2.1 Spatial Topology Model

In the standard geometry, straight line Euclidean distances within a closed 1.0×1.0 unit square are computed. Because this domain has boundaries, points near the edge experience the edge effect that restricted the possible location of neighbor points. In contrast, wraparound geometry is the model that assign the torus topology connecting the opposite sides of a square. In this case, there are no boundaries, so all points will have the equivalent geometric positional characteristics. Moreover, wrapping shift is possible the shortest pass is likely to form crossing spatial boundaries, allowing for what is a wrapping movement.

2.2 Algorithm Complexity

Naïve algorithm computes the distance for all pairs of (i, j) , requiring number of $N(N-1)$ computation. This method has the advantage of no dependencies between iterations, making it readily to parallelize around the outer loop. Besides, the optimization algorithm computes the distance only in case $j > i$, using the symmetric of distance $d_{ij} = d_{ji}$. It can close computational load to half $N(N-1)/2$, however a race condition may occur in a parallel environment because computed distances should be updated to the distance list of minimum and maximum for each i and j .

3. Code Structure

3.1 Distance Computation Function (distFunc)

The function which calculates the distance between p_1 and p_2 is Incarnated differently following the geometric structure.

- calcDist: Use standard Euclidean distance equation
- calcDistWrap: Afterwards computing absolute value between the coordinates for the operation of wraparound, when the value

exceeds 0.5, perform correction logic.

This process spends more CPU cycles than standard distance computation with including extra if statement and computation of absolute values.

3.2 Parallelized Structure of Naïve Algorithm

There are no dependencies between the outer loop in naïve algorithm, it is parallelized simply via `#pragma omp parallel for` statement.

- Parallel domain declaration:

```
#pragma omp parallel for reduction(+:totalNearest,  
totalFurthest) schedule(static)  
for (int i = 0; i < n; ++i) {...}
```

- Reduction: Since the totalNearest and totalFurthest variables must be summed by multiple threads simultaneously, the reduction(+:...) clause is used to sum safely without a race condition.
- Scheduling: Since the workload of each iteration is consistent with N distance, the load between threads is distributed evenly adopting schedule(static) which has the least overhead.

3.3 Optimization Algorithm and Memory Management

Optimization algorithm (analyzePointsOptimized) utilizes a symmetric, so the information of p_i and p_j should be updated simultaneously when we compute $d(p_i, p_j)$. Advanced strategies are employed to prevent to that each different thread accesses to equivalent p_j in the parallel environment.

- Thread local storage: To avoid a race condition, respective localMin and localMax are assigned for each thread instead of using Atomic. The performance is retained eliminating synchronization cost against that increasing the memory usage to $O(N \times ThreadNum)$.
- Triangular loop and scheduling: Inner loop forms for(int j = i + 1; j < n; ++j), so the triangular loop structure is formed where the smaller i, the larger computation and the bigger i, the less computation.
 - Static: When the index is splitted simply, the performance is degraded concentrating loads on the front threads.
 - Dynamic/Guided: As assignment of operation chunk dynamically on runtime using schedule(dynamic) or schedule(guided), this load imbalance is resolved.
- Merge phase: After that completes computation, after the computation, a separate loop which decides the global maximum or minimum is executed to aggregate the scattered localMin/Max values from each thread.

4. Results

The experiment was conducted using 8 threads against N =100,000.

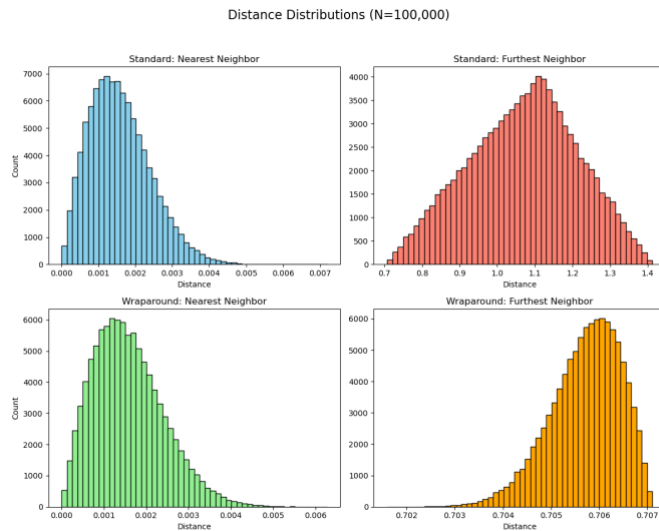


Figure 1: Distance distribution

Figure 1 shows the distribution of the nearest and furthest distance retained from standard geometry and wraparound geometry by uniform $N=1000,000$ points. At standard geometry, the nearest distances are concentrated on average about 0.0016 and the furthest distances are distributed from 0.7 to 1.4 widely. In contrast, at wraparound geometry, average distance of the nearest shows similar, but the furthest distances are concentrated on very narrow domain between 0.702 and 0.707.

4.1 Naïve Algorithm

Naïve algorithm has $N \times N$ rectangular loop domain and theoretically, perfectly uniform tasks are assigned to all threads.

4.1.1 Standard Geometry

In the standard geometry, because the simple Euclidean distance computation is performed, there are no conditional statements, and the computational cost is relatively low.

	Time spent (s)	Characteristic
Static	17.26	Best performance
Dynamic	18.82	Overhead (+9%)
Guided	17.95	+4%

[Table 1: Standard geometry]

The superiority of static scheduling arises from the iterative structure of the naïve algorithm. This algorithm has $N \times N$ shape rectangular loop and the amount of computation that is performed by each iteration is equivalent. In situations where the workload is perfectly uniform, static scheduling, which does not require redistribution of tasks at runtime and in fact, showed the shortest execution time.

In contrast, dynamic scheduling showed the result, about 1.5s slower. This can be interpreted the result accumulated task distribution overhead-lock

acquisition and queue access iteratively occurring at about 1 billion distance computation process entirely. Also, in terms of the memory access, static method that processes consecutive indices uniformly offers an advantage by utilizing data locality more efficiently.

4.1.2 Wraparound Geometry

In the wraparound environment, the conditional statements (if ($dx > 0.5$)...) are added to the distance calculation logic to handle periodic boundary conditions.

	Time spent (s)	Improvement for Naive
Static	43.68	2.53x
Dynamic	18.82	2.56x
Guided	17.95	2.24x

[Table 2: Wraparound geometry]

Increasing of computation cost is able to be attributed to the execution time for wraparound being more than twice as long as that for standard. It is not only because of adding floating points operations but because CPU pipeline is stalled repeatedly by branch misprediction is used to happen during the process that investigates whether boundaries are crossed based on coordinate differences. It is judged that the entire execution time increased significantly, cumulating the effect of this overhead.

Interestingly, I had expected that static scheduling would be the fastest, however, guided scheduling proved to be approximately 3.4s (about 8%) faster than static.

In the distance computation process, the frequency with which the conditional statement evaluates varies slightly depending on the point's position, and these differences make thread specific execution time jitter over billions of iterations. Because guided scheduling is the method that allocates large tasks first and breaks down the remaining task into smaller units, it absorbs these fine imbalances automatically. Consequentially, the entire execution time tended to give shorter than static.

4.2 Optimized Algorithm

We analyze the performance of the optimized algorithm, which halves the computational complexity $0.5N^2$ using symmetry ($d_{ij} = d_{ji}$).

	Time spent (s)	Improvement for Naive
Static	18.58	-7.65%
Dynamic	14.01	18.83%
Guided	15.12	12.48%

[Table 3: Optimized algorithm]

The performance degradation of static scheduling is based on the structural characteristic of the optimization algorithm. Even though the number of distance computation was reduced to half, the execution time was 21.96s, longer than the naïve (17.26s). This is because the loop structure has

triangular shape for $i < j$, caused severe load imbalance with tasks concentrating exceedingly on the thread handling low i values. On the other hand, this imbalance is resolved effectively by applying dynamic scheduling. Initial sections having much of high volume of computations are allocated uniformly as that each thread takes remained tasks dynamically, and as a result, the execution time decreased 16.65s surpassing naïve algorithm performance.

There are the apparent limitations to performance enhancement. Theoretically, computational loads are reduced 50%, however, the real execution time improved by only 4%. It is because the cache contention and the memory bandwidth saturation occurs from generating perpetual memory write in the process of updating the localMin and localMax arrays maintained per thread by the optimization algorithm. The memory bottleneck is conjectured to compensate much part of potential performance enhancement being able to retain the computation decreasing.

4.3 Impact of Thread Count

	Schedule	4 Threads	8 Threads	Speedup
Standard Naïve	Static	21.17s	17.26s	1.23x
Wraparound Naïve	Static	65.85s	43.68s	1.51x
Optimized	Dynamic	27.42s	14.01s	1.96x

[Table 4: Execution time of 4 threads vs 8 threads]

To evaluate the scalability of parallelization, the equivalent experiment was conducted in both 4 threads and 8 threads, and the execution time were compared. Since the hardware limitation, single thread experiments proved difficult, we focused on the relative speedup between 4 threads and 8 threads. Theoretically, doubling the number of threads is expected to halve execution time, resulting in a speedup of approximately 2.0.

4.3.1 Standard Geometry – Naïve Algorithm

In the standard geometry, the speed improvement was limited to 1.23 times, showing the lowest scalability among the three geometric structures. It is interpreted as being due to that standard distance computations that have lower computation complexity were limited a performance by memory bandwidth rather than CPU computation. On the other hand, even when increasing the number of threads, the rate at which data is supplied to the CPU acts as a bottleneck, interfering in parallel processing efficiency.

4.3.2 Wraparound Geometry – Naïve Algorithm

In the wraparound geometry, the speed improvement was 1.51 times, showing a higher scalability than standard geometry. Since, wraparound distance computation includes additional computations such as coordinate correction and in branching, it becomes compute-bound in nature. For this characteristic, the performance utilization is getting bigger with the addition of CPU cores, and parallel scalability could be seen to have improved relatively.

4.3.3 Optimized Algorithm – Dynamic Scheduling

In the optimized algorithm, in the case applying dynamic scheduling, the speed improvement was achieved a near-linear, 1.96 times. It is interpreted as the algorithmic advantage halving entire computational loads and the result of the dynamic scheduling effectively resolving load imbalance arising from the triangular loop.

5. Analysis

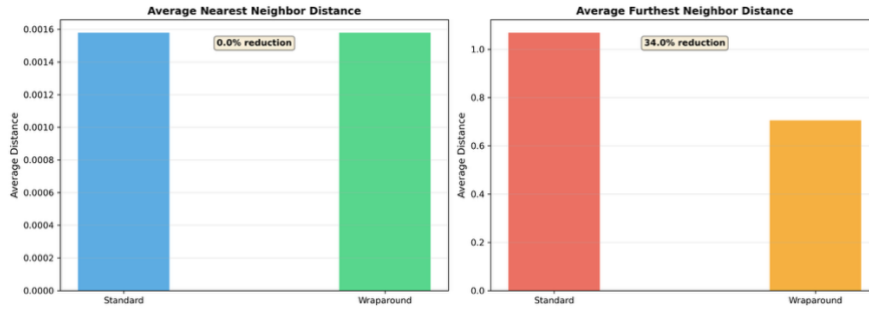


Figure 2: Average nearest and furthest distance

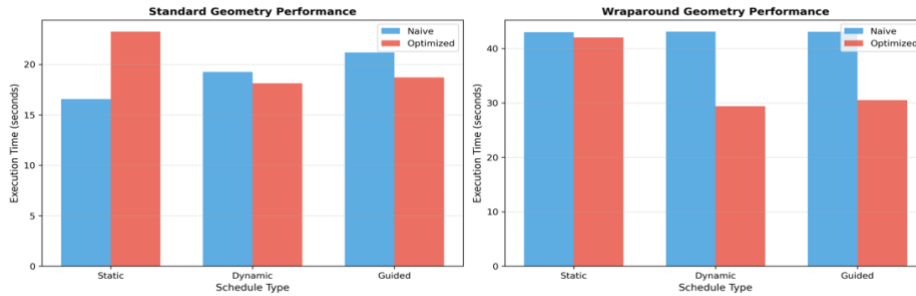


Figure 3: Execution time between standard and wraparound geometry

5.1 The influence of geometric structure on distance statistics

Figure 2 shows the results of comparing the average distance to the nearest neighbor and the furthest neighbor.

Both of two geometric maintain almost equivalent values with 0.0016. In the case of random uniform distribution, it means that the local clustering of points is not influenced much by effects of topological boundary. Whereas the furthest distance was decreased approximately 34% compared to standard at wraparound. At standard geometry, depending on the position of points, the distance is able to be increased to a maximum of $\sqrt{2}$, but in wraparound, the shortest path is formed across the boundary, thereby limiting of the maximum distance to approximately 0.707.

5.2 Performance and Scheduling efficiency

Figure 3 is the result comparing the execution time over each algorithm (naïve/optimized) and scheduling (static/dynamic/guided).

Wraparound geometry showed higher execution time than standard geometry consistently. This is because the coordinate correction operations for periodic boundary condition handling, the added if statements, and the resulting branch mispredictions cause the instruction pipeline to stall frequently.

Naïve algorithm has $N \times N$ uniform loop structure, and the workload of all iterations

are equivalent. This leads that static without runtime overhead performed the best. Because optimized algorithm is formed a triangular shape, since operations are concentrated in the small i interval, applying static scheduling causes significant variations in workload over threads, resulting in load imbalance. In some experiments, it was also observed that static-optimized performed slower than naïve.

Dynamic and guided scheduling resolved this imbalance successfully. In wraparound geometry, dynamic-optimized performed the fastest execution time, and it proves that applying algorithm optimization and pertinent scheduling together could lead actual performance enhancements. Furthermore, even though optimized algorithm was decreased to half for computational load, performance improvements are limited. It is able to be interpreted as being due to that during the process of updating the localMin/Max array, memory writes become concentrated, leading to cache contention and memory bandwidth bottlenecks which limited performance.

6. Conclusion

Through this study, the following conclusion is deduced regarding the parallelization of the N-body problem.

6.1 Naïve Algorithm and Static Scheduling

In $O(N^2)$ problem that has uniform workload, static scheduling provides the best performance without synchronization overhead. Application of a complex scheduling method is even able to arouse a performance degradation.

6.2 Management of Load Imbalance

The optimization algorithm utilizing symmetric involves load imbalance inevitably. In this case, static scheduling degrades the performance severely, and guided or dynamic scheduling is essential. Specifically, in the loop structure decreasing overload gradually, guided scheduling is superior in terms of theoretically and empirically.

6.3 Computational Cost and The Utility of Optimization

In the case with low unit computation cost, maintaining a simple access pattern may be more advantageous than the optimization reducing the computation cost. In contrast, in the case with high unit computation cost, advantages of the optimization algorithm become clear.

Consequently, when we write high performance parallel code, must not merely focus on reducing theoretical computation complexity, but also consider the resulting changes in loop structure and memory access patterns to select a pertinent OpenMP scheduling strategy.